BUA
BADR UNIVERSITY IN ASSUIT
جامعة بـــدر بأسيـوط

SEPTEMBER 24, 2026

LAB 1: GETTING STARTED

INTERNET OF THINGS - STUDENT PRACTICE SHEET

IOT TEACHING TEAM
SCHOOL OF AI & DATA MANAGEMENT

IoT - Lab 1 - Student Practice Sheet

Practical Student Lab Guide: Getting Started with ESP32, Arduino IDE 2.0, & Wokwi

Course: Internet of Things (IoT-401) | Term: Fall 2026–2027 | Lab Session: 120 Minutes

Student Name: _____	Student ID: _____
Lab Section / Bench #: _____	Date: _____

1. Lab Objective & Theoretical Grounding

Atomic Practical Goal

In this lab, you will set up the Arduino IDE 2.0 development environment, configure board support for the ESP32 microcontroller, write and upload your first C++ "Hello World" program to toggle an LED, monitor serial output at 115200 Baud, and test circuit logic virtually using the Wokwi online simulator.

Theoretical Alignment: Mapped to Level 1 (Physical Devices) and Level 2 (Connectivity & Toolchain) of Rayes & Salam's 4-Level IoT Reference Framework.

2. Workstation Equipment & Materials Checklist

Equipment / Tool	Specification	Verification Check
ESP32 Development Board	ESP32-DevKitC / NodeMCU-32S (3.3V Logic)	[ ] Verified Present
Micro-USB Cable	Data-capable USB-to-MicroUSB cable	[ ] Connected to PC
Host Workstation	PC running Windows/Linux/macOS with Arduino IDE 2.0	[ ] Launched
Browser Simulator	Wokwi Online Simulator (wokwi.com)	[ ] Browser Ready

3. Step-by-Step Practical Lab Tasks

Task 1: Configuring Arduino IDE 2.0 for ESP32

1. Launch Arduino IDE 2.0 on your workstation.
2. Open File -> Preferences (or press Ctrl + Comma).
3. Locate "Additional boards manager URLs" and paste the following URL: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
4. Click "OK" to save preferences.
5. Open the Boards Manager from the left-hand icon panel (or press Ctrl + Shift + B).
6. Search for "esp32" and locate "esp32 by Espressif Systems". Click INSTALL (ver 2.0.9+).

Task 2: Board Selection & Hardware Port Connection

1. Connect your ESP32 board to the PC using the micro-USB cable.
2. In Arduino IDE, go to Tools -> Board -> esp32 -> select "ESP32 Dev Module".
3. Go to Tools -> Port -> select the COM port corresponding to your connected ESP32 (e.g., COM3, COM4, or /dev/ttyUSB0).

Task 3: Writing & Uploading "Hello World" LED Blink

Type the following C++ code into your Arduino IDE editor:

ESP32 LED Blink Sketch (sketch.ino)

```
#define LED_PIN 2   // Built-in LED on GPIO 2

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("ESP32 Lab 1 Ready!");
}

void loop() {
  digitalWrite(LED_PIN, HIGH);
  Serial.println("LED State: ON");
  delay(1000);
  digitalWrite(LED_PIN, LOW);
```

```
  Serial.println("LED State: OFF");
  delay(1000);
}
```

1. Click the "Verify" button (Checkmark icon) to compile the sketch and verify zero syntax errors.

2. Click the "Upload" button (Right Arrow icon) to flash the program onto your ESP32.

3. Note: If the terminal displays "Connecting...", press and hold the physical "BOOT" button on the board until flashing begins.

4. Open Tools -> Serial Monitor and set the baud rate dropdown to 115200 Baud to view live terminal messages.

Task 4: Simulating ESP32 Projects in Wokwi Browser Simulator

1. Open your web browser and navigate to https://wokwi.com.

2. Under "Start New Project", click on "ESP32".

3. In the sketch.ino window, paste your LED Blink C++ code.

4. Click the green "Start Simulation" button to test code execution inside the browser.

5. Click the "+" button on the visual canvas to add a Red LED and 220 Ohm resistor. Wire pin D26 to LED Anode, Cathode to Resistor, and Resistor to GND. Modify code to define LED_PIN as 26 and verify.

Task 5: Experimental Variations (Frequency & Serial Toggling)

Variation A: Change the delay(1000) value to delay(200) for fast blinking (5 Hz). Re-upload and observe.

Variation B: Modify the Serial.println statement to log custom telemetry: "Station X: Temperature Normal, LED Toggled".

4. Instructor / TA Demonstration Sign-Off Matrix

Milestone	Demonstration Requirement	Status	TA Signature
Check 1	Arduino IDE 2.0 ESP32 Board Manager	[ ] Verified	

	Installed		
Check 2	Physical ESP32 Built-in LED Blinking (1 Hz)	[ ] Verified	
Check 3	Serial Monitor Output Verified at 115200 Baud	[ ] Verified	
Check 4	Wokwi Browser Simulation Running Successfully	[ ] Verified	
Check 5	Experimental Fast-Blink Variation (5 Hz) Executed	[ ] Verified	

5. Post-Lab Reflection Questions & Student Workspace

Question 1

Explain the functional difference between setup() and loop() in an Arduino C++ sketch.

[Student Response]

Question 2

Why is the ESP32 microcontroller superior to traditional Arduino Uno for building real-world IoT systems?

[Student Response]

Question 3

What is the purpose of adding the Espressif Board Manager URL into Arduino IDE Preferences?

[Student Response]

Question 4

Describe two advantages of using the Wokwi online simulator prior to assembling physical hardware.

[Student Response]